

Генетические и меметические алгоритмы для решения задачи TSP

Пучков Пётр

7 мая 2026 г.

Аннотация

Настоящая работа посвящена применению генетических и меметических алгоритмов к NP-трудной задаче коммивояжёра (TSP). В ходе работы будут изучены различные виды мутаций, скрещивания и отбора. Кроме того будут рассмотрены гибридные методы, сочетающие в себе генетические алгоритмы и локальную оптимизацию. Изученные алгоритмы будут реализованы и протестированы. На основе полученных во время тестирования данных, будет проведён сравнительный анализ эффективности выбранных эвристик.

1 План работы

1. Изучение основ.

- Прочитаю рекомендованные главы 7–9 в Handbook of Metaheuristics, при необходимости – дополнительные источники.
- Сделаю краткий обзор основных подходов к решению: точные, эвристические, метаэвристические.

2. Освоение генетических и меметических алгоритмов.

- Разберу базовые компоненты генетического алгоритма: представление решений, популяция, селекция, скрещивание, мутация.
- Пойму, как работает меметический алгоритм, т.е. как добавляется локальный поиск для улучшения каждого решения.

3. Обзор и выбор операторов.

- Изучу известные операторы селекции, скрещивания, мутации и локального поиска, применяемые для TSP.

4. Программная реализация.

- Реализую алгоритм вместе с архитектурой, позволяющую легко заменять эвристики.

5. Экспериментальное сравнение.

- Запущу алгоритмы на тестовых наборах данных, измерю качество решений, время работы, скорость сходимости.
- Построю графики и таблицы, проанализирую, как разные операторы и параметры влияют на результат.

2 Введение

Задача коммивояжёра (Traveling Salesman Problem, **TSP**) - это одна из классических задач комбинаторной оптимизации, цель которой — найти кратчайший или самый дешёвый маршрут, проходящий через все заданные пункты (города) ровно по одному разу и возвращающий в исходный пункт. Задача **TSP** входит в число **NP**-трудных задач.

Для решения подобных задач не существует алгоритмов, работающих за полиномиальное время. Поэтому для их решения применяются метаэвристические подходы - они позволяют за сравнительно небольшое время получить результат, сравнимый с наилучшим решением. В данной работе будут рассмотрены генетические алгоритмы (**GA**), имитирующие процессы биологической эволюции. В отличие от классических алгоритмов оптимизации, в **GA** оптимизируется не одно решение, а набор решений, который называется популяцией. В рамках **GA** решения (маршруты) подвергаются операциям скрещивания, мутации и отбора, что позволяет эффективно исследовать пространство состояний.

К сожалению, классические реализации **GA** часто сталкиваются с проблемой «преждевременной сходимости» (отсутствие разнообразия) и низкой точности при поиске в локальных областях. Для решения этой проблемы используются меметические алгоритмы **MA**, которые дополняют генетический поиск процедурами локального улучшения.

В работе рассмотрены наиболее популярные алгоритмы скрещивания, мутации, отбора. Данные алгоритмы реализованы на языке Kotlin. Также проведён сравнительный анализ их скорости и эффективности при использовании в различных сочетаниях и разными параметрами.

3 Техническая часть

3.1 Формальная постановка задачи

Дан граф $G(V, E)$ вместе с функцией $\omega : E \rightarrow \mathbb{R}_+$, которая задаёт веса на рёбрах. Необходимо найти наименьший по длине гамильтонов цикл.

Мы будем рассматривать менее общую задачу, которая тоже является **NP**-трудной. А именно, в нашей задаче вершинами графа будут точки на плоскости, а длинами рёбер - евклидовы расстояния между ними.

3.2 Принципы работы генетических алгоритмов

В TSP алгоритм оптимизирует популяцию - набор из нескольких гамильтоновых циклов, выполняя несколько итераций оптимизации. Стоит отметить, что в таком случае не одно решение движется по пространству решений, а набор решений, который движется по пространству популяций. На каждой итерации согласно оператору селекции выбираются решения, которые будут подвержены скрещиванию, а также выбираются решения, которые будут удалены из популяции. При таком подходе за небольшое количество итераций может произойти вырождение популяции - все решения станут практически идентичными. Чтобы избежать этой проблемы, после скрещивания к каждому решению с некоторой вероятностью применяется оператор мутации. Такие итерации повторяются до тех пор, пока не будет достигнуто условие остановки.

3.3 Механизмы контроля популяции

С точки зрения динамики, в **GA** эволюционирует не отдельный индивид, а распределение вероятностей перехода из одной популяции в другую. Можно выделить две основные схемы управления поколениями.

- Классический **GA**. В начале каждой итерации (поколения) формируется промежуточная популяция одинакового с текущей размера путём применения оператора селекции. Затем к отобранным особям применяются рекомбинация и мутация, в результате чего создаётся полностью новое поколение. При такой схеме часть лучших решений может теряться, поэтому часто используется *элитизм*: несколько лучших особей гарантированно переносятся в следующее поколение без изменений.
- Стационарный (монотонный) **GA**. На каждом шаге вместо создания целого поколения генерируется один (или небольшое число)

потомков. Новый индивид оценивается и помещается в популяцию, при этом обычно удаляется наихудшая особь текущей популяции. Это обеспечивает монотонное неухудшение рекорда, увеличивает селективное давление, но быстрее снижает разнообразие. Реализация эффективна при хранении популяции в виде кучи, что позволяет быстро находить и заменять худшего индивида.

3.4 Оператор селекции (отбора)

Целью оператора селекции является выбор из популяции решений, которые будут далее подвержены скрещиванию. Существует несколько основных операторов отбора.

3.4.1 Пропорциональная селекция

Данный оператор селекции предложен одним из основоположников ГА, Дж. Холландом в рамках классического ГА. Обозначим приспособленность (значение оптимизируемой функции) i -ого решения за f_i и среднюю приспособленность за $\bar{f}$. При пропорциональной селекции i -ое решение выбирается для селекции с вероятностью, пропорциональной приспособленности, т.е. с вероятностью $\frac{f_i}{\bar{f}}$. Стоит отметить, что здесь подразумевается, что приспособленность максимизируется, что неверно для суммарной длины цикла в задаче TSP. Поэтому в качестве f_i предлагается рассматривать величину, обратную длине цикла.

У данного метода есть существенные недостатки:

- При наличии решения с высокой приспособленностью высока вероятность скорого вырождения популяции
- Наоборот, на поздних этапах работы алгоритма, когда популяция близка к искомому результату, особи имеют почти одинаковую приспособленность, поэтому сходимость значительно замедлится

3.4.2 Турнирная селекция

Недостатки пропорциональной селекции демонстрируют, что при селекции стоит учитывать не точное значение приспособленности, а только факт того, какое решение более приспособлено. Данный подход применяется в турнирной селекции - самом распространённом операторе отбора. Простейший вариант турнирной селекции — бинарный турнир: случайно выбираются две особи из популяции, сравниваются их значения целевой функции, и в промежуточную популяцию помещается лучшая из двух.

Такой турнир повторяется, полностью заполняя промежуточную популяцию.

Для устранения погрешностей случайного семплирования применяют несмещённый турнир: генерируется случайная перестановка индексов популяции π , после чего последовательно проводятся состязания между особями с индексами π_i и π_{i+1} . Такой приём гарантирует, что каждый индивид примет участие ровно в двух турнирах, снижая шум отбора.

Для контроля над давлением отбора вводится параметр $0.5 \leq S \leq 1$, который задаёт вероятность выбора лучшей особи. При проведении турнира между двумя особями с вероятностью S в итоговую популяцию будет взята более приспособленная особь, а иначе - менее приспособленная. При этом реальное селективное давление составляет $2S$. Значение $S = 0.5$ соответствует полностью случайному выбору (давление 1), $S = 1$ — максимальному давлению 2.

3.5 Оператор скрещивания (кроссовера)



Оператор скрещивания моделирует рекомбинацию генов. Он совмещает два родительских решения с целью получить одного или нескольких потомков, наследующих полезные признаки обоих родителей. Для задачи коммивояжёра решение представляет собой перестановку городов (гамильтонов цикл), поэтому стандартные операторы, работающие с бинарными строками, непосредственно неприменимы. Тем не менее для задачи TSP были как разработаны специальные операторы скрещивания, так и адаптированы некоторые стандартные операторы.

В рамках работы реализованы и экспериментально сравниваются несколько классических операторов скрещивания для задачи TSP, описанных в литературе [?].

3.5.1 Стандартный однородный кроссовер, адаптированный для перестановок

Имея две последовательности одинаковой длины проще всего скомбинировать их - случайно. На каждой позиции i с вероятностью $1/2$ выбрать из какого родителя взять элемент. Но в случае комбинации перестановок результат может перестать быть перестановкой. Поэтому предлагается выбирать город и помещать в потомка, если он ещё не был использован; в противном случае позиция временно остаётся пустой. После первого прохода все незаполненные позиции заполняются оставшимися городами случайно. Такой метод сохраняет часть относительного порядка городов, но слабо учитывает соседство вершин в туре, поэтому его эффективность для TSP ограничена.

3.5.2 Двухточечный кроссовер (Order Crossover, OX)

Более естественным для задачи коммивояжёра считается оператор OX (Order Crossover), предложенный специально для перестановок [?]. Процедура выполняется следующим образом:

1. Выбираются две случайные точки разреза i и j ($1 \leq i < j \leq n$).
2. Потомок наследует фрагмент тура между этими точками непосредственно от первого родителя (города на позициях $i, \dots, j$ копируются без изменений).
3. Начиная с позиции $j + 1$, оставшиеся позиции тура заполняются городами второго родителя, взятыми в порядке их следования во втором родителе, начиная с той же позиции $j + 1$, но пропуская те города, которые уже присутствуют в потомке. При достижении конца перестановки просмотр продолжается циклически с начала.

Данный оператор сохраняет не абсолютные позиции городов, а их относительный порядок (циклическую последовательность) и фрагменты маршрута, что хорошо согласуется с природой TSP.

3.5.3 PX EAX ???

3.6 Оператор мутации

Мутация вносит небольшие случайные изменения в решения-потомки, выполняя две основные функции: во-первых, она может случайно улучшить особь, действуя подобно локальному поиску малой глубины; во-

вторых, она препятствует преждевременной утрате разнообразия в популяции, без которой алгоритм может сойтись к посредственному локальному оптимуму, далёкому от глобального [?].

В контексте задачи коммивояжёра оператор мутации заключается в локальной перестройке тура. Мы рассматриваем и сравниваем в экспериментальной части следующие варианты мутации, представляющие собой один или несколько локальных ходов.

3.6.1 Обмен двух городов (Swap)

Простейший оператор: случайно выбираются два различных индекса i и j в перестановке, и соответствующие города меняются местами. Это соответствует одной транспозиции и является классическим примером мутации для задач на перестановках [?].

3.6.2 Инверсия участка маршрута (Inversion / 2-opt move)

Выбираются два индекса $i < j$, и порядок городов в подпоследовательности от i до j меняется на противоположный. Эта операция эквивалентна одному шагу локального поиска **2-opt**, а потому может приводить к существенному улучшению тура, если удалены два пересекающихся ребра и восстановлена допустимая структура цикла. В меметических алгоритмах подобная мутация часто заменяется систематическим применением **2-opt** или **3-opt** ко всем потомкам [?].

3.6.3 Перемешивание подпоследовательности (Scramble)

Случайно выбранный непрерывный фрагмент тура подвергается случайной перестановке городов внутри него. Этот оператор вносит большее разнообразие, чем одиночный обмен или инверсия, и помогает преодолевать локальные барьеры, когда популяция находится на плато.

3.6.4 Параметры мутации

Интенсивность мутации обычно регулируется вероятностью применения оператора к потомку или долей генов, подвергаемых изменению. Следуя [?], мы задаём *частоту мутации* p_m , которая определяет среднюю долю позиций, затрагиваемых мутацией. При $p_m = 5\%$ это означает, что примерно $p_m \cdot n$ элементов перестановки будут случайным образом представлены (например, путём выполнения $\lfloor p_m \cdot n/2 \rfloor$ случайных обменов).

Более высокое значение p_m замедляет сходимость, но позволяет популяции дольше сохранять генетическое разнообразие и, в конечном счёте, находить более качественные решения.

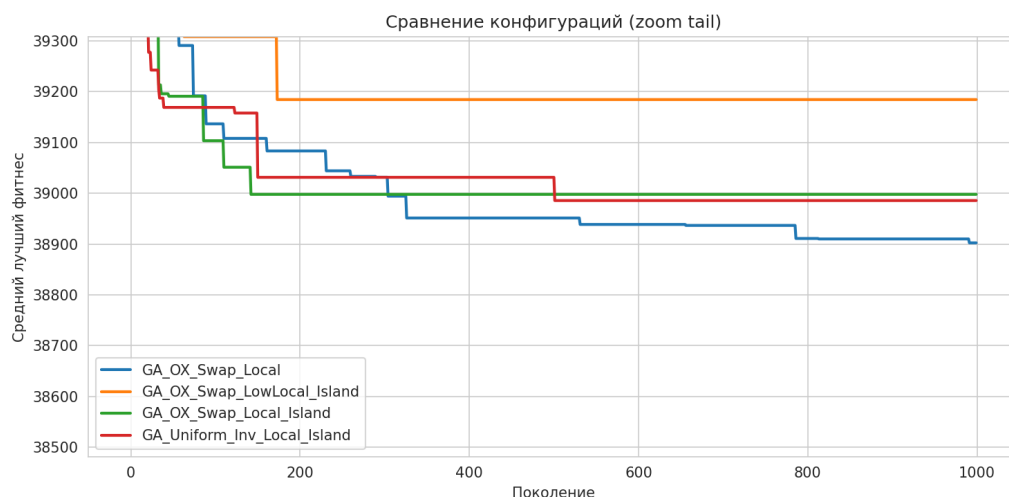
3.7 Меметические алгоритмы

Меметический алгоритм (МА) расширяет генетический алгоритм, встраивая локальную оптимизацию потомков (например, 2-opt в TSP) [?]. Обычно применяется *ламарковская модель*: улучшенное решение заменяет исходное, что быстро повышает качество популяции. Платой за это является риск преждевременной сходимости: интенсивный локальный поиск может свести всю популяцию в один бассейн притяжения. Для сохранения баланса используют частичный ламаркизм (улучшаются не все особи), запрет дубликатов и адаптивное управление глубиной поиска.

В задаче коммивояжёра МА эффективно сочетает глобальное исследование, обеспечиваемое популяционными операторами, с локальной доводкой решений, что позволяет достигать высокого качества даже на крупных экземплярах [?].

4 Реализация алгоритма

5 Экспериментальное исследование



6 Заключение

7 Литература